Загадка Эйнштейна как онтология

Пример к лекции 3: классы, свойства, замыкание мира, машина вывода

О. М. Атаева А. П. Халов

Осень 2026

Загадка

На улице стоят пять домов. Каждый дом окрашен в свой цвет. В каждом живёт человек своей национальности, у каждого своё животное, свой напиток и своя марка сигарет.

1. Англичанин живёт в красном доме.
2. У испанца есть собака.
3. В зелёном доме пьют кофе.
4. Украинец пьёт чай.
5. Зелёный дом стоит сразу справа от дома цвета слоновой кости.
6. Тот, кто курит Old Gold, разводит улиток.
7. В жёлтом доме курят Kool.
8. В центральном доме пьют молоко.
9. Норвежец живёт в первом доме.
10. Сосед того, кто курит Chesterfield, держит лису.
11. В доме по соседству с тем, где держат лошадь, курят Kool.
12. Тот, кто курит Lucky Strike, пьёт апельсиновый сок.
13. Японец курит Parliament.
14. Норвежец живёт рядом с синим домом.

Кто пьёт воду? Кто держит зебру?

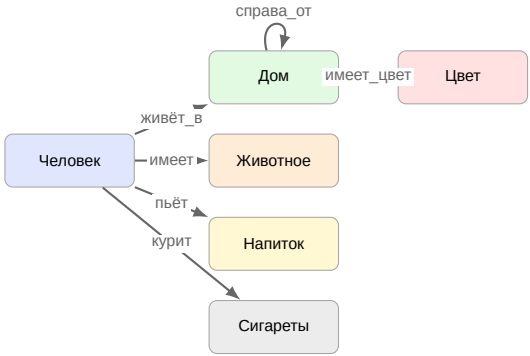


Главное

Загадка известна с 1962 года. Все пятнадцать утверждений записываются конструкциями OWL, ответ находит машина вывода. В оригинале пятый дом цвета слоновой кости, ivory.

Замысел записи

- Шесть классов: Дом, Животное, Человек, Напиток, Сигареты, Цвет. По пять индивидов в каждом.
- Шесть свойств: живёт_в, имеет, курит, пьёт, имеет_цвет, находится_справа_от.
- Пятнадцать утверждений записываются как принадлежность индивида классу, заданному выражением.
- Замыкание мира: каждый класс равен перечислению своих пяти индивидов, у каждого человека есть дом, животное, сигареты и напиток.



Главное

Та же схема, что в онтологии примера редактора: классы, свойства, индивиды, ограничения. Новое здесь одно: ответ целиком вычисляет машина вывода.

Классы и индивиды

```
class Дом(Thing): pass
class Животное(Thing): pass
class Человек(Thing): pass
class Напиток(Thing): pass
class Сигареты(Thing): pass
class Цвет(Thing): pass
AllDisjoint([Дом, Животное, Человек,
             Напиток, Сигареты, Цвет])

дома = [Дом(f"дом{i}") for i in range(1, 6)]
люди = [Человек(n) for n in ("англичанин",
   "испанец", "норвежец", "украинец", "японец")]
for группа in (дома, животные, люди,
               напитки, сигареты, цвета):
    AllDifferent(группа)
```

- Шесть классов попарно не пересекаются: дом не есть сигареты и не есть цвет. Без этого машина вывода вправе считать один индивид и домом, и напитком.
- Внутри класса индивиды различны: англичанин не есть испанец. OWL по умолчанию этого не знает, два имени могут обозначать одного и того же.
- Тридцать индивидов, по пять в классе. Пока это открытый список: шестой дом не запрещён.

Главное

Два объявления, которые в примере редактора выглядели формальностью, здесь решают исход: дизъюнктность классов и различность индивидов.

Свойства

```
class живёт_в(ObjectProperty, FunctionalProperty,
              InverseFunctionalProperty):
    domain = [Человек]; range = [Дом]
class имеет(...):          Человек -> Животное
class курит(...):          Человек -> Сигареты
class пьёт(...):           Человек -> Напиток
class имеет_цвет(...):     Дом -> Цвет
class находится_справа_от(...):  Дом -> Дом

class обитаем(ObjectProperty):
    inverse_property = живёт_в
class находится_слева_от(ObjectProperty):
    inverse_property = находится_справа_от
```

- Каждое свойство функционально: у человека один дом. И обратно функционально: в доме один человек. Вместе это взаимно однозначное соответствие.
- Обратные свойства с именами (обитаем, пьётся, курится, окрашивает, находится_слева_от) нужны для утверждений, где субъектом становится дом или напиток.
- Область определения и область значений задают, какие классы свойство связывает.

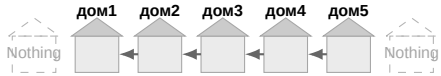
Главное

Условие «у каждого своё» записано свойствами: функциональность и обратная функциональность дают взаимно однозначное соответствие между классами.

Порядок домов

```
дом2.находится_справа_от = дом1
дом3.находится_справа_от = дом2
дом4.находится_справа_от = дом3
дом5.находится_справа_от = дом4

дом1.is_a.append(
    находится_справа_от.only(Nothing))
дом5.is_a.append(
    находится_слева_от.only(Nothing))
```



- Четыре факта задают цепочку из пяти домов, стрелка на рисунке есть находится_справа_от.
- Про крайние дома сказано отдельно: слева от первого и справа от пятого ничего нет. `Nothing` есть пустое множество, предопределённое в OWL.
- Без этих двух строк машина вывода вправе поставить рядом ещё один дом, шестой.

Главное

Открытый мир: чего нет в записи, то неизвестно. Отсутствие записывается явно, ограничением на пустое множество.

Утверждение как принадлежность классу

1. Англичанин живёт в красном доме.

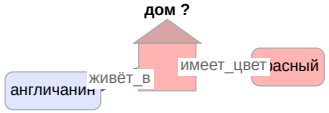
```
англичанин.is_a.append(
    живёт_в.some(имеет_цвет.value(красный)))
```

2. У испанца есть собака.

```
испанец.имеет = собака
```

3. В зелёном доме пьют кофе.

```
кофе.is_a.append(пьётся.some(
    живёт_в.some(имеет_цвет.value(зелёный))))
```



- Фраза становится выражением класса: англичанин есть тот, кто живёт в доме с красным цветом.
- Прямой факт записывается значением свойства, это короче.
- Когда субъект фразы напиток, запись идёт через обратное свойство: кофе пьётся человеком из зелёного дома.

Главное

Каждое утверждение загадки есть фраза «X есть тот, у кого ...»: выражение класса, приписанное индивиду.

Соседство: «слева или справа»

10. Сосед того, кто курит Chesterfield, держит лису.

```
def сосед_с(условие):
    return (находится_справа_от.some(условие())
          | находится_слева_от.some(условие()))

Chesterfield.is_a.append(курится.some(живёт_в.some(
    сосед_с(lambda: обитаем.some(имеет.value(лиса))))))
```

14. Норвежец живёт рядом с синим домом.

```
норвежец.is_a.append(живёт_в.some(
    сосед_с(lambda: имеет_цвет.value(синий))))
```



слева или справа

- «Рядом» значит справа или слева: объединение двух ветвей.
- Читается с конца: дом с лисой, дом рядом с ним, человек в нём, его сигареты.
- Самые длинные выражения загадки, по шесть вложенных ограничений.

Главное

Объединение классов записывает «или», вложенные ограничения записывают цепочку связей.

Прямые факты: значение свойства

2. У испанца есть собака.

```
испанец.имеет = собака
```



4. Украинец пьёт чай.

```
украинец.пьёт = чай
```



9. Норвежец живёт в первом доме.

```
норвежец.живёт_в = дом1
```



13. Японец курит Parliament.

```
японец.курит = Parliament
```

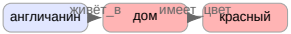


Четыре утверждения называют конкретную связь двух индивидов. В Protégé это поле Facts, в Owlready2 присваивание. Выражение класса здесь не нужно.

Человек и его дом: «живёт в доме, у которого ...»

1. Англичанин живёт в красном доме.

```
англичанин.is_a.append(
 живёт_в.some(имеет_цвет.value(красный)))
```



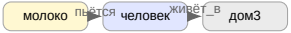
7. В жёлтом доме курят Kool.

```
Kool.is_a.append(курится.some(
 живёт_в.some(имеет_цвет.value(жёлтый))))
```



8. В центральном доме пьют молоко.

```
молоко.is_a.append(
 пьётся.some(живёт_в.value(дом3)))
```



Субъект есть человек или то, что он курит и пьёт. Запись идёт по цепочке: человек → дом → цвет или номер дома. Утверждение 8 использует значение дом3, утверждение 1 значение цвета.

Запись через обратное свойство

3. В зелёном доме пьют кофе.

```
кофе.is_a.append(пьётся.some(
 жи-
вёт_в.some(имеет_цвет.value(зелёный))))
```



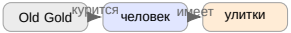
5. Зелёный дом стоит сразу справа от дома цвета слоновой кости.

```
зелёный.is_a.append(окрашивает.some(
 находится_справа_от.some(
  имеет_цвет.value(слоновая_кость))))
```



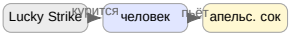
6. Тот, кто курит Old Gold, разводит улиток.

```
OldGold.is_a.append(
 курится.some(имеет.value(улитки)))
```



12. Тот, кто курит Lucky Strike, пьёт апельсиновый сок.

```
LuckyStrike.is_a.append(
 курит-
ся.some(пьёт.value(апельсиновый_сок)))
```



Субъект фразы напиток, сигареты или цвет. У них нет прямых свойств к человеку, поэтому запись начинается с обратного свойства: пьётся, курится, окрашивает. Дальше цепочка как в предыдущей группе.

Соседство: «справа или слева»

10. Сосед того, кто курит Chesterfield, держит лису.

```
Chesterfield.is_a.append(курится.some(
 живёт_в.some(сосед_с(
  lambda:
обитаем.some(имеет.value(лиса))))))
```



11. В доме по соседству с тем, где держат лошадь, курят Kool.

```
Kool.is_a.append(курится.some(
 живёт_в.some(сосед_с(
  lambda:
обитаем.some(имеет.value(лошадь))))))
```



14. Норвежец живёт рядом с синим домом.

```
норвежец.is_a.append(живёт_в.some(
 сосед_с(lambda:
имеет_цвет.value(синий))))
```



«Рядом» означает справа или слева, запись есть объединение двух ветвей через or. Самые длинные выражения загадки: до шести вложенных ограничений. Дом с лисой или лошадью задаётся через обитаем: дом, обитаемый человеком, у которого есть лиса.

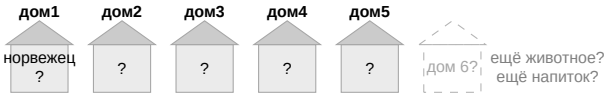
Сводка: пятнадцать утверждений в одной таблице

№	Фраза	Запись
1	англичанин живёт в красном доме	англичанин ∈ живёт_в some (имеет_цвет value красный)
2	у испанца есть собака	испанец имеет собака
3	в зелёном доме пьют кофе	кофе ∈ пьётся some (живёт_в some (имеет_цвет value зелёный))
4	украинец пьёт чай	украинец пьёт чай
5	зелёный дом справа от дома цвета слоновой кости	зелёный ∈ окрашивает some (находится_справа_от some (имеет_цвет value слоновая_кость))
6	курит Old Gold, разводит улиток	OldGold ∈ курится some (имеет value улитки)
7	в жёлтом доме курят Kool	Kool ∈ курится some (живёт_в some (имеет_цвет value жёлтый))
8	в центральном доме пьют молоко	молоко ∈ пьётся some (живёт_в value дом3)
9	норвежец живёт в первом доме	норвежец живёт_в дом1
10	сосед курящего Chesterfield держит лису	Chesterfield ∈ курится some (живёт_в some сосед(обитаем some (имеет value лиса)))
11	рядом с домом, где лошадь, курят Kool	Kool ∈ курится some (живёт_в some сосед(обитаем some (имеет value лошадь)))
12	курит Lucky Strike, пьёт апельсиновый сок	LuckyStrike ∈ курится some (пьёт value апельсиновый_сок)
13	японец курит Parliament	японец курит Parliament
14	норвежец рядом с синим домом	норвежец ∈ живёт_в some сосед(имеет_цвет value синий)

Утверждение «пять домов» записано порядком домов. Четыре утверждения есть прямые факты, десять записаны выражениями классов.

Запуск без замыкания мира

- Машина вывода запускается на тридцати индивидах и пятнадцати утверждениях. Противоречий нет.
- Выводов почти нет. Про испанца известно, что он имеет собаку, это и было записано. Кто что пьёт, кто где живёт, машина не сообщает.
- Причина та же, что в примере редактора: открытый мир. Записано, что есть пять домов, и нигде не записано, что других домов нет. Записано, что у испанца собака, и нигде не записано, что у каждого человека есть животное.



кто	дом	животное	сигареты	напиток
англичанин	?	?	?	?
испанец	?	собака	?	?
норвежец	дом1	?	?	?
украинец	?	?	?	чай
японец	?	?	Parliament	?

Главное

В открытом мире пятнадцать утверждений ответа не дают: из записанного он не следует, пока не запрещены лишние дома, животные и напитки.

Замыкание мира

```
Дом.equivalent_to.append(OneOf(дома))
Животное.equivalent_to.append(OneOf(животные))
Человек.equivalent_to.append(OneOf(люди))
Напиток.equivalent_to.append(OneOf(напитки))
Сигареты.equivalent_to.append(OneOf(сигареты))
Цвет.equivalent_to.append(OneOf(цвета))

Человек.is_a.append(
    живёт_в.some(Дом) & имеет.some(Животное)
  & курит.some(Сигареты) & пьёт.some(Напиток))
```



- Класс равен перечислению: дома это только дом1 … дом5. Шестого нет.
- У каждого человека есть дом, животное, сигареты и напиток.
- Пять напитков делятся между пятью людьми без остатка, каждому ровно один.

Главное

Замыкание мира есть два объявления: класс равен списку индивидов, у каждого индивида заполнены все связи. В лекции 2 это делала `close_world`.

Ответ машины вывода



цвета домов: жёлтый, синий, красный, слоновая кость, зелёный

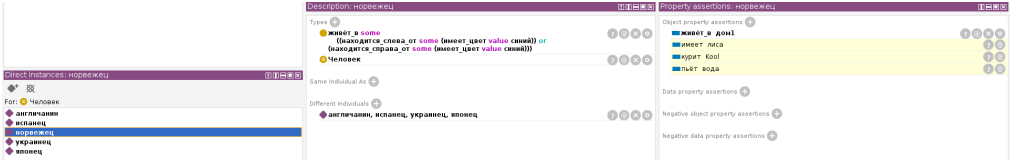
Pellet через Owlready2: 2,7 секунды, противоречий нет. Воду пьёт норвежец, зебру держит японец. Ни одна клетка рисунка не записана целиком: каждая выведена из пятнадцати утверждений и замыкания.

- Машина вывода даёт объяснение каждого вывода: цепочку аксиом, из которых он следует.
- Файл `zebra.owl` открывается в Protégé: вкладка Individuals, запуск Pellet, выведенные связи выделены цветом.

Главное

Ответ получен без единой строки процедурного кода. Знание записано, машина вывода нашла единственную модель, в которой все утверждения верны.

Ответ в Protégé: норвежец



Файл `zebra.owl`, Protégé 5.6.1, машина вывода Pellet, флажок Show Inferences включён. Записано только `живёт_в дом1` и тип с условием соседства. На жёлтом фоне выведено: `имеет лиса`, `курит Kool`, `пьёт вода`. Кнопка «?» у каждой строки открывает объяснение.

Главное

Без флажка Show Inferences выведенные связи скрыты, без замыкания мира их нет. Вариант Pellet (Incremental) в этой версии Protégé даёт ошибку в журнале, нужен обычный Pellet.

Что показывает пример

1. Фразы естественного языка переводятся в выражения классов: «тот, кто ...» есть класс, индивид приписывается к нему.
2. Дизъюнктность классов и различность индивидов в OWL не подразумеваются, их записывают явно.
3. Функциональность и обратная функциональность свойств записывают взаимно однозначное соответствие.
4. Открытый мир: то, чего нет в записи, считается неизвестным. Отсутствие записывается ограничением на `Nothing`.
5. Замыкание мира делается перечислением индивидов класса и обязательными связями. До замыкания машина вывода молчит, после него находит единственное решение.
6. В примере редактора те же средства: выводимый класс «Готовая к решению статья» наполняется после замыкания, проверка данных остаётся за SHACL.

Главное

Загадка решена только средствами OWL. Это показывает силу машины вывода и цену открытого мира: без явного замыкания ответа нет.